Experiment 17: Docker Volumes and Port Mapping

1. Lab Objective

Understand how **data persistence** works in containers using **volumes** and how applications inside containers are exposed using **port mapping** in **Docker**.

By the end of this lab, participants will:

- Understand **container storage limitations**
- Use **Docker volumes** for persistent storage
- Map **container ports to host ports**
- Verify persistence after container restart

---

2. Prerequisites

Ensure the following are available:

- **Docker** installed and running
- Basic knowledge of Docker commands
- Internet connection
- Command line access

Verify Docker installation:

```
docker --version
```

---

3. Key Concepts

Concept	Description
Volume	Persistent storage managed by Docker
Bind Mount	Mapping host directory to container
Port Mapping	Exposing container port to host
Persistence	Data survives container deletion

---

4. Why Persistence is Needed

Containers are **ephemeral**.

If a container is deleted:

Container Data → Lost

To solve this, **Docker volumes** store data **outside the container filesystem**.

Application

│

Container

│

Docker Volume

│

Host Storage

---

5. Step 1 – Pull a Sample Application Image

Download a web server container.

docker pull nginx

The image comes from **Docker Hub** and contains the **NGINX** web server.

---

6. Step 2 – Run Container with Port Mapping

Run a container and expose its port.

docker run -d -p 8080:80 nginx

Explanation:

Parameter	Meaning
-d	Run container in background
-p	Map ports
8080	Host port
80	Container port

Architecture:

Browser

│

localhost:8080

│

Host Machine

│

Container Port 80

Test in browser:

http://localhost:8080

You should see the **NGINX welcome page**.

---

7. Step 3 – View Running Containers

Check running containers:

```
docker ps
```

Example output:

```
CONTAINER ID   IMAGE   PORTS
abcd1234       nginx   0.0.0.0:8080->80/tcp
```

---

8. Step 4 – Create a Docker Volume

Create a volume for persistent storage.

```
docker volume create my-volume
```

Verify volumes:

```
docker volume ls
```

Example output:

```
DRIVER    VOLUME NAME
local     my-volume
```

---

9. Step 5 – Run Container with Volume Mapping

Run container using the created volume.

```
docker run -d \
-p 8081:80 \
-v my-volume:/usr/share/nginx/html \
nginx
```

Explanation:

Parameter	Purpose
-v	Mount volume
my-volume	Docker volume
/usr/share/nginx/html	Container directory

This directory stores the **NGINX web content**.

---

10. Step 6 – Verify Volume Mapping

Inspect the container.

```
docker inspect <container_id>
```

Look for the **Mounts section** showing the attached volume.

---

11. Step 7 – Test Data Persistence

Create a sample HTML file inside the container.

Access the container shell:

```
docker exec -it <container_id> bash
```

Create a file:

```
echo "Hello Persistent Storage!" > /usr/share/nginx/html/index.html
```

Open in browser:

http://localhost:8081

Output:

Hello Persistent Storage!

---

12. Step 8 – Restart Container

Stop the container.

docker stop <container_id>

Remove the container:

docker rm <container_id>

Run a new container using the same volume:

docker run -d -p 8081:80 -v my-volume:/usr/share/nginx/html nginx

Visit again:

http://localhost:8081

The same content appears because **data is stored in the volume**.

---

13. Docker Volume vs Container Storage

Feature	Container Storage	Docker Volume
Data persistence	No	Yes
Managed by Docker	No	Yes
Survives container deletion	No	Yes

---

14. Important Commands Summary

Command	Purpose
docker volume create	Create volume
docker volume ls	List volumes
docker run -p	Port mapping
docker run -v	Volume mapping
docker inspect	View container details

---

15. Demo Flow (Recommended)

1️⃣ Run NGINX container with **port mapping**
2️⃣ Access app in browser
3️⃣ Create **Docker volume**
4️⃣ Run container using volume
5️⃣ Add data inside container
6️⃣ Delete container
7️⃣ Recreate container
8️⃣ Show **data persistence**

This clearly demonstrates **why volumes are important in real DevOps deployments**.

---

16. Real-World DevOps Use Cases

Docker volumes are used in applications such as:

Application	Use Case
Databases	Store persistent data
CMS systems	Save uploaded files
Logs	Persist application logs
Configuration	Store config files

Examples:

- **MySQL** containers
- **PostgreSQL** containers
- **MongoDB** containers

---

17. Expected Lab Outcome

After completing this experiment, participants will:

- Understand **Docker port mapping**
- Create and use **Docker volumes**
- Implement **persistent storage for containers**
- Manage container data effectively in **Docker**